



Pontificia Universidad Javeriana
Facultad de Ingenieria
Fundamentos de Ingenieria de Software

Informe de Postmortem

Reporte del ciclo, retrospectiva Starfish y PIP

Milestone 1: Base Funcional del Sistema

Equipo SYNTAX TECH

Profesor	Luis Gabriel Moreno Sandoval
Monitor	Juan Pablo Arias Buitrago
Fecha	19 de mayo de 2026
Repositorio	https://github.com/puj-course/FIS_2610_3517_G4

*Documento construido a partir del postmortem original del milestone, sin recortar informacion,
adaptado al formato institucional de la plantilla suministrada.*

Integrantes

Nombre	GitHub	Rol Scrum / tecnico
Sebastian Ramirez Maldonado	@Sarm-m	Scrum Master
Sebastian Vargas	@juanvargax	Product Owner / Sprint Planner
Samuel Freile	@samuelfl680	Configuration Manager
Solon Losada	@solonlosada2006	DevOps Engineer
Sebastian Rodriguez Ramirez	@juserora	QA Lead

Tabla 1: Integrantes y roles del equipo SYNTIX TECH.

Tabla de contenido

Integrantes	1
1. Introduccion	4
2. Postmortem – Milestone 1: Base Funcional del Sistema	5
2.1. 1. Resumen Ejecutivo	5
2.2. 2. Contexto de Ingeniería y Negocio	5
2.3. 3. Revisión de Datos del Proceso y Calidad	5
2.3.1. Desempeño real vs. planeado	5
2.3.2. Lecciones aprendidas	6
2.4. 4. Reporte de Roles	6
2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)	6
2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juanvargax)	6
2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelff680)	6
2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)	7
2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006) .	7
2.5. 5. Identificación de Causa Raíz	7
2.6. 6. Retrospectiva	7
2.7. 7. Retrospectiva Starfish	8
2.7.1. Seguir haciendo Seguir Haciendo	8
2.7.2. Comenzar a hacer Comenzar a Hacer	8
2.7.3. Dejar de hacer Dejar de Hacer	9
2.7.4. Mas de Más de	9
2.7.5. Menos de Menos de	9
2.8. 8. Plan de Mejora para el Milestone 2	10
2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem	10
2.9.1. 9.1 Revisión del producto, esfuerzo y proceso	10
2.9.2. 9.2 Evaluación objetiva de roles	12
2.9.3. 9.3 Reporte del ciclo por responsabilidades	14
2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad	14

2.9.5. 9.5 Administración de configuración, riesgos y reutilización 15

2.9.6. 9.6 Reflexión del trabajo en grupo 15

2.9.7. 9.7 Verificación Starfish: respuestas por eje 15

2.9.8. 9.8 PIP - Plan de Mejora del Proceso 16

1. Introduccion

Este informe presenta el postmortem del Milestone 1 del proyecto DriveControl / AutoMinder Enterprise, desarrollado por el equipo SYNTIX TECH en el marco de la asignatura Fundamentos de Ingenieria de Software. El documento conserva de forma integral la informacion del postmortem original y la reorganiza bajo el formato institucional suministrado, manteniendo el contenido de resumen ejecutivo, contexto, revision de calidad, evaluacion de roles, causa raiz, retrospectiva Starfish, plan de mejora y respuestas completas a las preguntas guia de la presentacion de postmortem.

El proposito de este reporte es dejar una evidencia clara del ciclo: que producto se produjo, que esfuerzo se invirtio, que proceso se siguio, que problemas aparecieron, cuales fueron sus causas, que aprendizajes surgieron y que acciones de mejora deben incorporarse en el ciclo siguiente o en proyectos futuros. La estructura facilita la lectura academica y permite defender el trabajo con trazabilidad metodologica, tecnica y de calidad.

2. Postmortem – Milestone 1: Base Funcional del Sistema

Proyecto: Drive Control / Syntix Tech **Stack:** MERN (MongoDB, Express, React, Node.js)

Equipo:

Nombre	Usuario	Rol
Sebastián Ramírez Maldonado	@Sarm-m	Scrum Master
Samuel Freile	@samuelff680	Configuration Manager
Sebastián Rodríguez Ramírez	@Juserora	Quality Assurance Lead
Solón Losada	@solonlosada2006	DevOps Engineer
Sebastián Vargas	@juanvargax	Product Owner / Sprint Planner

2.1. 1. Resumen Ejecutivo

El Milestone 1 tuvo como objetivo establecer la base funcional del sistema sobre el stack MERN, asegurando el flujo de trabajo colaborativo mediante GitFlow, e implementando los módulos iniciales de autenticación (Login/Registro) y la estructura visual del Dashboard. El equipo logró entregar estos componentes, aunque con retrasos atribuibles a una subestimación del esfuerzo real y a la carga académica concurrente del calendario de parciales universitarios. Las lecciones de este hito sentaron las bases metodológicas y técnicas para los sprints siguientes.

2.2. 2. Contexto de Ingeniería y Negocio

El objetivo de negocio en este primer hito era demostrar la viabilidad técnica de la plataforma mediante el acceso controlado de usuarios y la visualización de la interfaz principal. Técnicamente, el reto consistió en alinear al equipo alrededor de un stack nuevo para varios de sus miembros y en establecer convenciones de trabajo colaborativo desde cero.

La prioridad del sprint fue el núcleo de autenticación segura y el Dashboard base. Sin embargo, la ausencia de un criterio de aceptación técnico unificado (Definition of Done) generó ambigüedad sobre cuándo una historia de usuario podía considerarse realmente terminada, lo que impactó directamente la velocidad de entrega.

2.3. 3. Revisión de Datos del Proceso y Calidad

2.3.1. Desempeño real vs. planeado

- Las historias de usuario relacionadas con Login y Registro fueron completadas, pero con un tiempo de ciclo mayor al estimado, principalmente por retrabajos derivados de rutas del frontend rotas al integrar código sin validación previa de compilación.
- El Dashboard base fue entregado en su versión mínima, sin alcanzar algunos criterios visuales inicialmente planteados.

- No se utilizó un criterio de aceptación formalizado, lo que generó discrepancias entre lo que cada integrante consideraba "listo" lo que el Product Owner esperaba.

2.3.2. Lecciones aprendidas

- La estimación de esfuerzo debe contemplar la capacidad real del equipo, incluyendo compromisos académicos externos.
 - Integrar código directamente sobre el repositorio sin verificar la compilación local es una fuente recurrente de deuda técnica inmediata.
 - La falta de un DoD unificado es equivalente a no tener criterios de calidad: cada miembro trabajó con su propia interpretación de "terminado".
-

2.4. 4. Reporte de Roles

2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)

Durante este primer milestone, el Scrum Master enfrentó el reto de consolidar un equipo que aún estaba alineando sus ritmos de trabajo y sus herramientas de comunicación. Las Daily Standups no se realizaron con la frecuencia ni la consistencia requeridas, lo que generó silos de información entre los integrantes. Se logró mantener la estructura de GitFlow activa desde el inicio, lo cual fue un acierto significativo. Para el siguiente ciclo, se debe formalizar el canal de comunicación principal (Discord) y establecer una agenda mínima para las sesiones sincrónicas, garantizando que los impedimentos se escalen oportunamente y no bloqueen el avance del equipo por períodos prolongados.

2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juan-vargax)

La planeación del sprint inicial careció de amortiguadores de tiempo frente a la carga académica externa del equipo. Las historias de usuario fueron estimadas con optimismo sin considerar la capacidad real disponible en la semana de parciales. El valor de negocio entregado fue parcial: el flujo de autenticación quedó funcional, pero el Dashboard base no alcanzó los criterios visuales definidos en la refinación inicial. Para los siguientes milestones, el Sprint Planner debe incorporar un factor de capacidad real como insumo obligatorio de la planeación, y las historias de usuario deben descomponerse en sub-tareas técnicas verificables antes de iniciar el sprint.

2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelf680)

El Configuration Manager estableció exitosamente la estructura de GitFlow desde el inicio del proyecto, lo cual fue fundamental para evitar conflictos mayores durante la integración de ramas. Sin embargo, se detectaron casos en los que los integrantes subieron código con estados incompletos o realizaron merges sin revisión previa, lo que introdujo inestabilidades en la rama principal. Para el siguiente ciclo, se debe implementar una política explícita de protección de

ramas y una checklist de pre-merge que impida integrar código sin que el flujo de compilación local haya sido verificado previamente.

2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)

La ausencia de un Definition of Done unificado fue el principal hallazgo de calidad en este milestone. Sin un criterio claro de aceptación, las historias de usuario fueron marcadas como completas sin pasar por una verificación funcional estructurada. Se identificaron rutas del frontend rotas tras la integración de código no validado, lo que hubiera podido prevenirse con una checklist de pre-commit básica. Para el Milestone 2, el QA Lead debe liderar la definición formal del DoD del equipo y establecer un proceso mínimo de revisión funcional antes de cerrar cualquier historia de usuario.

2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006)

En este primer milestone, el rol de DevOps se concentró en sentar las bases del entorno de desarrollo compartido. No se contó con un pipeline de integración continua activo, lo que permitió que código con errores de rutas llegara a la rama principal sin detección automática. La infraestructura de desarrollo fue configurada manualmente por cada integrante, generando inconsistencias entre entornos. Para el siguiente ciclo, se debe priorizar la configuración de al menos un paso básico de validación automática (build check) que actúe como primer filtro antes de que cualquier rama sea integrada.

2.5. 5. Identificación de Causa Raíz

La causa raíz principal de los problemas del Milestone 1 fue la **ausencia de un Definition of Done unificado y verificable**, combinada con la **integración de código sin validación previa de compilación local**. Estas dos condiciones operaron de forma sinérgica: sin un DoD claro, no había un estándar que obligara a verificar la compilación antes de integrar, y sin esa verificación, los errores de rutas del frontend llegaron al repositorio compartido, generando retrabajos que consumieron tiempo del sprint.

Como causa secundaria, la **estimación de esfuerzo no consideró la capacidad real del equipo**, al ignorar la carga académica concurrente del calendario de parciales universitarios. Esto resultó en compromisos de sprint que el equipo no pudo honrar dentro del tiempo disponible.

2.6. 6. Retrospectiva

El primer sprint del proyecto fue, en esencia, un proceso de autodescubrimiento como equipo. Cada integrante llegó con supuestos distintos sobre cómo se trabajaría, qué herramientas se usarían y cuán rápido fluiría el desarrollo. La realidad del sprint reveló que esos supuestos no estaban alineados: mientras algunos miembros avanzaban de forma autónoma, otros esperaban coordinación que nunca llegó de forma explícita.

El mayor aprendizaje no fue técnico sino metodológico: un equipo sin criterios compartidos de "terminado" no puede entregar con consistencia, independientemente de las habilidades individuales de sus miembros. La deuda metodológica acumulada en este hito fue tan costosa en tiempo como los propios errores técnicos.

En términos positivos, la adopción de GitFlow desde el inicio fue una decisión que evitó conflictos de integración mayores. La autonomía individual de los integrantes permitió avanzar en paralelo, y la comunicación a través de Discord, aunque irregular, funcionó como canal de desbloqueo para dudas puntuales. Estos son los cimientos sobre los cuales el equipo puede construir una disciplina más sólida en los milestones siguientes.

El equipo reconoce que la carga académica externa es una variable real que no puede ignorarse en la planeación. Incorporarla como factor explícito en la estimación de capacidad no es una excusa, sino una práctica de ingeniería de software responsable y realista.

2.7. 7. Retrospectiva Starfish

Reglas aplicadas: cada frase inicia con un verbo en infinitivo; ninguna frase se repite entre ejes.

2.7.1. Seguir haciendo Seguir Haciendo

#	Frase	Justificación
1	Mantener la estructura de GitFlow activa desde el inicio del proyecto.	Permitió aislar el trabajo de cada integrante en ramas propias y evitó conflictos directos en la rama principal durante el sprint.
2	Fomentar la autonomía individual de los integrantes para avanzar en paralelo.	Varios módulos pudieron desarrollarse de forma simultánea gracias a que cada miembro tenía claridad sobre su tarea asignada.
3	Usar Discord como canal principal de comunicación asíncrona del equipo.	Permitió resolver dudas puntuales y desbloquear a integrantes sin necesidad de convocar reuniones formales en cada ocasión.
4	Consolidar el núcleo técnico del stack MERN antes de agregar funcionalidades secundarias.	Establecer primero la autenticación y el Dashboard base como prioridades evitó dispersar el esfuerzo del equipo en features no críticas.
5	Dividir las historias de usuario en sub-tareas técnicas atómicas desde la planeación.	Facilita la asignación, el seguimiento y la verificación individual de cada componente del sprint.

2.7.2. Comenzar a hacer Comenzar a Hacer

#	Frase	Justificación
1	Definir y documentar un Definition of Done unificado antes de iniciar cada sprint.	La ausencia de este criterio fue la causa raíz principal de los problemas de calidad e integración del milestone.
2	Implementar una checklist de pre-commit que incluya verificación de compilación local.	Hubiera prevenido la integración de código con rutas rotas que generó retrabajos durante el sprint.
3	Establecer una política explícita de "1 issue = 1 cambio. ^{en} el repositorio.	Permite trazabilidad directa entre cada modificación del código y la tarea que la originó.

#	Frase	Justificación
4	Planificar la capacidad real del equipo considerando la carga académica del periodo.	La semana de parciales no fue contemplada en la estimación inicial, lo que resultó en compromisos que el equipo no pudo cumplir.
5	Documentar las rutas de la API y los contratos de interfaz antes de comenzar el desarrollo.	Evita que diferentes integrantes asuman estructuras distintas que luego generan conflictos de integración.
6	Validar la compilación local de cada rama antes de abrir un Pull Request.	Es el control más básico y efectivo para evitar que errores triviales lleguen a la rama principal.

2.7.3. Dejar de hacer Dejar de Hacer

#	Frase	Justificación
1	Integrar código al repositorio sin haber verificado previamente que compila correctamente.	Esta práctica introdujo rutas del frontend rotas que bloquearon el avance de otros integrantes.
2	Subir ramas personales con estados incompletos o parcialmente funcionales sin avisarlo al equipo.	Genera confusión sobre el estado real del sprint y puede bloquear la integración de otras ramas dependientes.
3	Postergar la creación de issues en GitHub hasta después de haber comenzado el desarrollo.	Sin la issue creada, no hay trazabilidad del trabajo realizado ni forma de hacer seguimiento al progreso real del sprint.
4	Realizar merges sobre ramas principales sin revisión previa de al menos un integrante.	Un merge no revisado es una puerta abierta a errores que, una vez en la rama principal, afectan a todo el equipo.
5	Ignorar las prácticas metodológicas del equipo bajo el argumento del estrés académico.	El estrés externo es una razón para ajustar la capacidad del sprint, no para abandonar el proceso acordado.

2.7.4. Mas de Más de

#	Frase	Justificación
1	Aplicar rigor en los procesos de Scrum, especialmente en la planeación y el cierre del sprint.	La falta de rigor en este milestone fue directamente proporcional a los retrasos y a la deuda metodológica acumulada.
2	Revisar los criterios de aceptación de cada historia de usuario en la sesión de refinación.	Evita que el equipo avance sobre supuestos distintos acerca de qué significa que una historia está terminada.
3	Realizar Dailies sincronizadas con una agenda mínima definida.	Mejora la visibilidad del estado del sprint y permite escalar impedimentos antes de que se conviertan en bloqueos.
4	Generar trazabilidad de los sprints vinculando commits, issues y Pull Requests explícitamente.	Permite al equipo y al instructor reconstruir la historia del desarrollo y evaluar el proceso con datos objetivos.
5	Aterrizar la planeación del sprint a escenarios realistas con base en la velocidad real del equipo.	Las estimaciones optimistas son la principal causa de sprints incompletos y de frustración en el equipo.

2.7.5. Menos de Menos de

#	Frase	Justificación
1	Aplazar tareas pendientes al siguiente sprint sin un análisis explícito del impacto.	Cada tarea aplazada sin análisis transfiere deuda técnica o funcional que se acumula y complica los sprints siguientes.
2	Depender de reuniones sincrónicas para resolver dudas que podrían resolverse de forma asíncrona.	Las reuniones no planificadas consumen tiempo valioso del sprint y no siempre generan el resultado esperado.
3	Generar estimaciones optimistas que no contemplan riesgos ni tiempos de retrabajo.	El retrabajo por rutas rotas en este milestone consumió una fracción significativa del tiempo disponible.
4	Mantener ramas personales sin push durante períodos prolongados.	El código que no está en el repositorio no puede ser revisado, integrado ni respaldado por el equipo.
5	Buscar información de forma dispersa sin una referencia técnica centralizada y acordada por el equipo.	Genera soluciones heterogéneas al mismo problema y dificulta la coherencia técnica del codebase.

2.8. 8. Plan de Mejora para el Milestone 2

Acción	Responsable	Criterio de éxito
Definir y documentar el DoD del equipo antes del inicio del M2.	QA Lead (@Juserora)	DoD aprobado por todo el equipo en la sesión de planeación.
Implementar checklist de pre-commit con verificación de compilación local.	Configuration Manager (@samuelh680)	Ningún PR abierto sin checklist completada.
Planificar el sprint con factor de capacidad real (descontar parciales y compromisos).	Product Owner (@juanvargax)	Sprint planning incluye tabla de disponibilidad por integrante.
Configurar un build check básico en el pipeline de CI.	DevOps Engineer (@solonlosada2006)	El pipeline rechaza automáticamente ramas que no compilan.
Establecer política de 1 issue = 1 cambio y comunicarla al equipo.	Scrum Master (@Sarm-m)	Todos los commits del M2 referencian una issue activa.

2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem

Esta sección se agrega como matriz de verificación para dejar respondidas de forma explícita todas las preguntas guía de la presentación de Postmortem y Retrospectiva Starfish. Las respuestas se formulan con hechos del milestone, evitando evaluar personas y concentrándose en el desempeño del trabajo, el producto, el proceso y las oportunidades de mejora.

2.9.1. 9.1 Revisión del producto, esfuerzo y proceso

Pregunta guía de la PPT	Respuesta aplicada al Milestone 1: Base Funcional del Sistema
¿Qué producto se produjo?	Se produjo la base funcional del sistema: autenticación inicial con Login/Registro, estructura visual mínima del Dashboard y flujo colaborativo inicial con GitFlow.
¿Qué esfuerzo se invirtió para hacerlo?	El esfuerzo se concentró en levantar el stack MERN, coordinar ramas, entender el dominio y entregar las primeras pantallas. El tiempo real fue mayor al estimado por retrabajos, rutas rotas y carga académica externa.
¿Qué proceso se siguió para hacerlo?	Se trabajó con Scrum académico, asignación de historias, uso de ramas e integración en repositorio. El proceso todavía era inmaduro porque no existía un Definition of Done compartido ni validación local obligatoria antes de integrar.
¿Cómo fue el desempeño real comparado con lo planeado?	El desempeño real fue parcial frente a lo planeado: Login y Registro quedaron funcionales, pero el Dashboard se entregó en versión mínima y con criterios visuales pendientes. La velocidad real fue menor que la planeada.
¿Qué lecciones se aprendieron de esta experiencia?	El equipo aprendió que la planeación debe considerar capacidad real, que integrar sin compilar genera deuda inmediata y que una historia no puede cerrarse sin un criterio de aceptación común.
¿Debería usarse un criterio diferente, a nivel individual o de equipo, para el futuro?	Sí. Para el futuro se debe usar un DoD obligatorio por historia, con compilación local, revisión de rutas, evidencia visual mínima y validación del Product Owner antes de cerrar.
¿Dónde hay oportunidades para mejorar y por qué?	La mayor oportunidad está en formalizar el trabajo: dailies breves, issues atómicas, checklist de PR y definición de criterios de aceptación. Esto mejora trazabilidad y reduce retrabajo.
¿Dónde hubo problemas que deben corregirse para el próximo ciclo?	Se deben corregir la integración sin validación local, la falta de Dailies consistentes, la estimación optimista y la ausencia de criterios de aceptación verificables.
¿Qué hizo bien el equipo y en qué falló?	Hizo bien: adoptar GitFlow desde el inicio, avanzar en paralelo y construir el flujo base de autenticación. Falló en: estimar sin capacidad real, cerrar tareas con criterios ambiguos y depender de comunicación irregular.
¿Sirvieron las mejoras propuestas previamente?	Al ser el primer milestone, no existían mejoras previas formalizadas. Este reporte deja la línea base para evaluar si las mejoras sirven en el Milestone 2.
¿Cómo ha sido el desempeño comparado con otros equipos?	No se cuenta con datos verificables de otros equipos, por lo que no se afirma una comparación externa. La comparación válida es interna: este milestone funciona como línea base; los ciclos siguientes deben superar su nivel de trazabilidad, calidad y disciplina de cierre.
¿Se deben modificar los objetivos y las metas?	Sí. Las metas del siguiente ciclo deben ser menos amplias, más verificables y divididas en tareas técnicas pequeñas.

Pregunta guía de la PPT	Respuesta aplicada al Milestone 1: Base Funcional del Sistema
¿Cómo se deben modificar los procesos?	Incluir DoD, checklist de pre-commit/pre-PR, definición de capacidad real y una sesión de sincronización mínima para impedimentos.
¿Cuáles son las áreas de más alta prioridad para analizar?	Definition of Done, compilación local obligatoria, trazabilidad issue-commit-PR y planeación basada en disponibilidad real.

2.9.2. 9.2 Evaluación objetiva de roles

Rol	¿Qué funcionó?	¿Dónde estuvieron los problemas?	¿Dónde se puede mejorar?	Objetivo de mejoramiento para el Milestone 2
Scrum Master / Liderazgo	Se sostuvo una estructura inicial de trabajo y se mantuvo el uso de GitFlow.	Las dailies no fueron consistentes y los impedimentos no siempre se escalaron a tiempo.	Formalizar agenda mínima, canal principal y seguimiento de bloqueos.	Garantizar una rutina de comunicación breve y verificable durante todo el M2.
Product Owner / Planeación	Priorizó Login, Registro y Dashboard como base de valor.	La estimación no consideró parciales ni capacidad real.	Agregar factor de capacidad y criterios de aceptación claros.	Refinar cada historia con alcance, evidencia esperada y criterios de cierre.
Configuration Manager / Soporte	Definió la estructura de ramas y evitó conflictos mayores.	Hubo integraciones sin validación previa y ramas con estados incompletos.	Crear checklist de integración y política de ramas.	No aceptar PR sin evidencia de compilación local.
QA Lead / Calidad	Identificó rutas rotas y ambigüedad de cierre.	No existía proceso formal de inspección ni DoD.	Definir pruebas mínimas por historia.	Crear DoD y checklist funcional para todas las historias del M2.
DevOps Engineer / Proceso de entrega	Se configuraron bases de entorno de desarrollo.	No había CI básico para detectar fallos tempranos.	Agregar build check automático.	Incorporar una validación automática mínima antes de integrar.
Ingenieros / Desarrollo	El equipo logró implementar módulos iniciales en paralelo.	Cada integrante interpretó de forma distinta qué significaba terminar una tarea.	Trabajar con criterios de aceptación y pruebas locales antes de entregar.	Cerrar cada tarea con evidencia técnica y funcional.

2.9.3. 9.3 Reporte del ciclo por responsabilidades

Responsabilidad solicitada en la PPT	Respuesta del milestone
Resumen del ciclo	Ciclo de arranque que permitió construir base funcional, pero evidenció deuda de proceso en estimación, comunicación y cierre de calidad.
Liderazgo: motivación, compromisos, reuniones y apoyo requerido	La motivación se mantuvo, pero los compromisos se afectaron por carga académica y falta de seguimiento frecuente. Se requiere guía más cercana en DoD y criterios de aceptación.
Desarrollo: contenido producido frente a requisitos	El contenido producido cubrió autenticación y Dashboard mínimo; faltó mayor pulimiento visual y validación de rutas.
Planeación: tareas, seguimiento y compromisos	La planeación fue optimista y no incorporó amortiguadores por parciales, por lo que el seguimiento tuvo desviaciones.
Proceso: disciplina de trabajo, medición y seguimiento	El proceso existía de forma básica, pero sin medición real ni controles de calidad consistentes.
Calidad: estándares, inspecciones, defectos y PIP	La calidad fue funcionalmente aceptable para la base, pero con defectos evitables por falta de compilación local y DoD.
Soporte: logística, configuración, cambios y riesgos	GitFlow ayudó, pero la ausencia de checklist de configuración permitió integraciones inestables.
Reporte de ingenieros: planeado vs. ejecutado y mejora personal	Los ingenieros ejecutaron tareas principales, pero deben contrastar mejor lo planeado contra lo ejecutado y documentar evidencias de cierre.

2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad

Pregunta guía	Respuesta
¿La estrategia de desarrollo funcionó?	Funcionó parcialmente: empezar por autenticación y Dashboard fue correcto, pero la estrategia falló al no imponer controles mínimos de integración.
¿Qué otros enfoques hubieran sido más adecuados?	Hubiera sido más adecuado iniciar con una checklist técnica, contratos de rutas y DoD antes de implementar pantallas.
¿Cómo debería cambiarse la estrategia en el futuro?	Primero definir estándares de cierre, luego construir; cada historia debe incluir prueba local y evidencia antes de PR.
¿Cómo se tuvo en cuenta la usabilidad?	Se consideró mediante la estructura inicial del Dashboard, aunque quedó limitada en detalle visual.
¿Cómo se tuvo en cuenta la mantenibilidad?	GitFlow aportó orden, pero faltó separación más rigurosa de responsabilidades y documentación técnica.
¿Cómo se tuvo en cuenta la compatibilidad?	No se validó suficientemente entre entornos ni navegadores; debe incluirse como criterio futuro.

Pregunta guía	Respuesta
¿Cómo se tuvo en cuenta el desempeño?	No fue el foco del milestone; el desempeño se observó solo de forma manual.
¿Cómo se tuvo en cuenta la seguridad?	Se inició con autenticación, pero faltó fortalecer criterios de configuración y manejo de datos desde el inicio.
¿Cómo mejorar estos tópicos en el futuro?	Agregar criterios explícitos de usabilidad, mantenibilidad, compatibilidad, desempeño y seguridad en cada historia.

2.9.5. 9.5 Administración de configuración, riesgos y reutilización

Pregunta guía	Respuesta
¿Funcionó la administración de configuración?	Funcionó de forma básica por el uso de ramas, pero sin protección ni checklist formal.
¿Funcionó el control de cambios?	Funcionó parcialmente: hubo control por Git, pero no suficiente control de calidad antes del merge.
¿Cómo mejorarlos?	Usar ramas protegidas, checklist de PR, commits vinculados a issues y validación local obligatoria.
¿Fue efectivo el manejo y seguimiento de riesgos?	El manejo de riesgos fue reactivo; la carga académica y los errores de integración se detectaron tarde.
¿Fue buena la estrategia de reutilización?	Fue limitada porque el producto apenas iniciaba; debe crecer con componentes reutilizables del Dashboard.

2.9.6. 9.6 Reflexión del trabajo en grupo

Aspecto de reflexión	Respuesta
Comunicación del grupo	La comunicación por Discord sirvió para dudas puntuales, pero no reemplazó una rutina de seguimiento.
Calidad del trabajo de los integrantes	La calidad dependió demasiado del criterio individual.
Planeación de tareas y seguimiento	La planeación no midió disponibilidad real ni riesgo de retrabajo.
Mitigación de riesgos	Los riesgos se gestionaron después de aparecer, no antes.
Relación con el cliente / stakeholder académico	El valor entregado fue comprensible para el stakeholder académico, pero faltó evidencia más sólida de cumplimiento visual.
Relación con las tecnologías	El equipo empezó a alinearse con MERN, aunque con curva de aprendizaje evidente.

2.9.7. 9.7 Verificación Starfish: respuestas por eje

Eje de la estrella	Pregunta de la PPT que responde	Respuesta concreta del milestone
Comenzar a hacer	¿Qué acciones deberíamos comenzar para lograr el éxito, resolver problemas, mitigar riesgos, mejorar calidad, aprender, usar herramientas, distribuir tiempo y comunicar mejor?	Definir DoD, checklist de compilación local, política de 1 issue = 1 cambio y criterios de aceptación antes de desarrollar.
Más de	¿Qué acciones que ya hacemos deberíamos hacer más para mejorar productividad, comunicación, calidad y aprendizaje?	Hacer más revisión de criterios, dailies sincronizadas, trazabilidad y planeación con velocidad real.
Seguir haciendo	¿Qué acciones que ya hacemos bien deberíamos continuar?	Mantener GitFlow, autonomía controlada, Discord como apoyo y foco en el núcleo funcional.
Menos de	¿Qué acciones deberíamos hacer menos porque causan inconvenientes o reducen calidad, comunicación o planeación?	Reducir estimaciones optimistas, aplazamientos sin análisis y ramas personales sin push.
Dejar de hacer	¿Qué acciones deberíamos dejar de hacer porque no son productivas o ya no se necesitan?	Dejar de integrar sin compilar, cerrar historias sin evidencia y hacer merges sin revisión.

2.9.8. 9.8 PIP - Plan de Mejora del Proceso

Mejora priorizada	Problema que ataca	Cambio concreto en el proceso	Evidencia esperada
DoD por historia	Cierres ambiguos	Crear plantilla de cierre con criterios funcionales, técnicos y visuales	Historias cerradas con checklist completo
Validación local obligatoria	Rutas rotas e integración inestable	Exigir compilación y prueba manual antes del PR	Captura o comentario de validación en cada PR
Planeación por capacidad real	Sobrecarga por parciales	Registrar disponibilidad del equipo antes de comprometer alcance	Backlog ajustado a capacidad real
Trazabilidad issue-commit-PR	Dificultad para reconstruir decisiones	Referenciar issue en commits y PRs	Historial verificable en GitHub